📋 COMPLETE PROJECT DOCUMENTATION

AccessLayer

An AI-powered universal accessibility layer — automatic WCAG audits, real-time remediation, AI alt text, Indic voice control, and WhatsApp scorecards for any website on earth.

VERSION	TYPE	BACKEND	FRONTEND	DATABASE
2.0	Full-Stack SaaS	FastAPI + Python	React 19	Supabase / PostgreSQL

AI

Google Gemini

NAVIGATION

Table of Contents

#	SECTION	TOPIC
1	What Is This Project?	Project overview, mission, one-liner
2	The Problem	Web accessibility gap, legal risk, market context
3	The Solution	What AccessLayer does, core value proposition
4	Key Features	All 8 major features explained with visuals
5	How It Works	Architecture diagram, data flow, component map
6	User Journey	Step-by-step usage from sign-up to live widget
7	Tech Stack	Every technology used and why
8	API Reference	Backend routes, authentication, widget API
9	Deployment Guide	Render unified deployment — step by step
10	Environment Variables	Complete .env reference
11	Security	Auth model, SSRF protection, data safety
12	Use Cases	Who uses this and how

SECTION 1

What Is AccessLayer?

AccessLayer is a **drop-in SaaS platform** that makes *any website* accessible, WCAG-compliant, and voice-navigable — without the site owner touching their codebase — using a single <script> tag and AI.

1	100	12+	∞
Script tag to deploy	Max accessibility score	WCAG rule checks	Sites per account

🎯 The One-Liner

AccessLayer is **Grammarly for web accessibility** — it monitors your site continuously, tells you exactly what's broken, AI-fixes what it can, and gives visitors live assistive tools (voice, contrast, dyslexia mode) in any Indian language.

🌐 The Audience

Built for **Indian SMB website owners, developers, and agencies** who need WCAG compliance for ADA / EAA / RPwD Act coverage but don't have an in-house accessibility team or budget for a $50k audit firm.

SECTION 2

The Problem

Roughly 96.3% of the top one million websites fail basic accessibility checks (WebAIM Million, 2024). This is not a niche concern — it creates real legal exposure and excludes over a billion users with disabilities worldwide.

⚖️ Legal Risk

ADA lawsuits in the US crossed 4,600/year. India's RPwD Act 2016 and the EU's European Accessibility Act (EAA) 2025 mandate digital accessibility for businesses. Non-compliance = fines and litigation.

👷 Developer Cost

Manual WCAG audits cost ₹3–15 lakh per engagement. A single audit is a snapshot — violations creep back every sprint. No small dev shop can afford continuous expert review.

🌍 Exclusion Gap

India has 26.8M+ persons with disabilities. Screen-reader users, low-vision users, motor-impaired users, and non-English speakers are locked out of most Indian websites due to missing alt text, no keyboard nav, and English-only UIs.

Top WCAG Failures Found on Indian Websites

ISSUE	% OF SITES AFFECTED	IMPACT
Missing alt text on images	58%	Screen readers skip images entirely
Low color contrast text	83%	Low-vision users cannot read
Missing form labels	46%	Blind users cannot fill forms
No keyboard navigation	41%	Motor-impaired users are blocked
Missing document language	33%	Screen readers use wrong pronunciation
Empty buttons / links	29%	Assistive tech announces "button, button, button"

SECTION 3

The Solution

❌ Without AccessLayer

- ✓ ✗ Hire expensive consultant for one-time audit
- ✓ ✗ Fix issues manually across thousands of pages
- ✓ ✗ Violations return with every new feature
- ✓ ✗ No alerts when score drops
- ✓ ✗ Voice control requires native app rebuild
- ✓ ✗ No Hindi / regional language support

VS

✅ With AccessLayer

- ✓ 1 script tag — deploys in 60 seconds
- ✓ Continuous automated WCAG audits, no consultants
- ✓ AI generates alt text for every image automatically
- ✓ WhatsApp + email alerts when score drops
- ✓ Voice control for visitors in Hindi, Tamil, Telugu & more
- ✓ Live dashboard with trend charts and fix tips

1	2	3	4	5
Dashboard Audit	**AI Fix Tips**	**Embed Widget**	**Live Alerts**	**Voice Control**
Server-side WCAG + SEO scan of your URL. Score from 0–100. Full violation list with WCAG criterion.	One-click Gemini AI explanation: why it matters, exact corrected HTML, WCAG reference.	Copy 2-line script tag. Widget runs in-browser, scans live DOM, injects accessibility toolbar.	Automatic WhatsApp + email when accessibility score drops ≥5 points.	Visitors speak in Hindi / English. Sarvam STT + Gemini understand commands, execute on page.

SECTION 4

Key Features

WCAG Accessibility Audit Engine

Server-side audit fetches the live HTML of any URL, runs 12+ WCAG 2.1 AA checks (missing alt, low contrast, no form labels, keyboard traps, empty buttons, lang attribute, skip links, etc.) and returns an accessibility score 0–100 with per-violation impact levels (critical / serious / moderate / minor).

Critical: score −15 | Serious: score −10 | Moderate: score −5 | Minor: score −2

AI-Generated Alt Text (Google Gemini)

For any image on a monitored site, the widget can call the backend which sends the image bytes to Gemini Vision and returns a concise, accurate alt-text sentence (≤18 words). Results are cached to avoid repeat API calls. The widget injects the generated alt into the live DOM without changing the original site's code.

Indic Voice Control (Sarvam AI + Gemini)

The embedded widget captures microphone input, sends audio to Sarvam AI for speech-to-text in Hindi, Tamil, Telugu, Kannada, Malayalam, Bengali, Marathi, and Hinglish. The transcript goes to Gemini which maps the intent to one of: CLICK, FILL_FORM, SCROLL, READ_TEXT, GO_BACK, SET_MODE. The widget executes the action on the live DOM.

```
"संपर्क बटन दबाओ" → CLICK #contact-button
"scroll down" → SCROLL direction:down
"high contrast on" → SET_MODE contrast
```

In-Browser Accessibility Toolbar (widget.js)

A floating toolbar injected into visitor's browser by the widget script. Controls include:

High Contrast
Inverts colors for low-vision users

Dyslexia Font
Switches to OpenDyslexic font

Read Aloud
TTS reads selected/page text

Bigger Text
+2 font-size steps globally

Focus Highlight
Visible keyboard focus ring

Voice Nav
Hindi/English voice commands

Dashboard — Sites, Audits, Trend Charts

Authenticated dashboard shows all monitored sites with their latest accessibility score, SEO score, total audits run, and a trend sparkline. Per-site detail view shows full audit history, violation breakdown by severity, shareable public report links, and the embed code snippet.

WhatsApp + Email Alerts

If accessibility score drops ≥5 points vs the previous audit, the system automatically fires a WhatsApp message (via Fast2SMS) and/or an email scorecard (via Emergent Email API) to the site owner. The WhatsApp bot also acts as an interactive concierge — owners can message their site URL to get an instant audit and fix tips via Gemini AI.

Authentication — Clerk

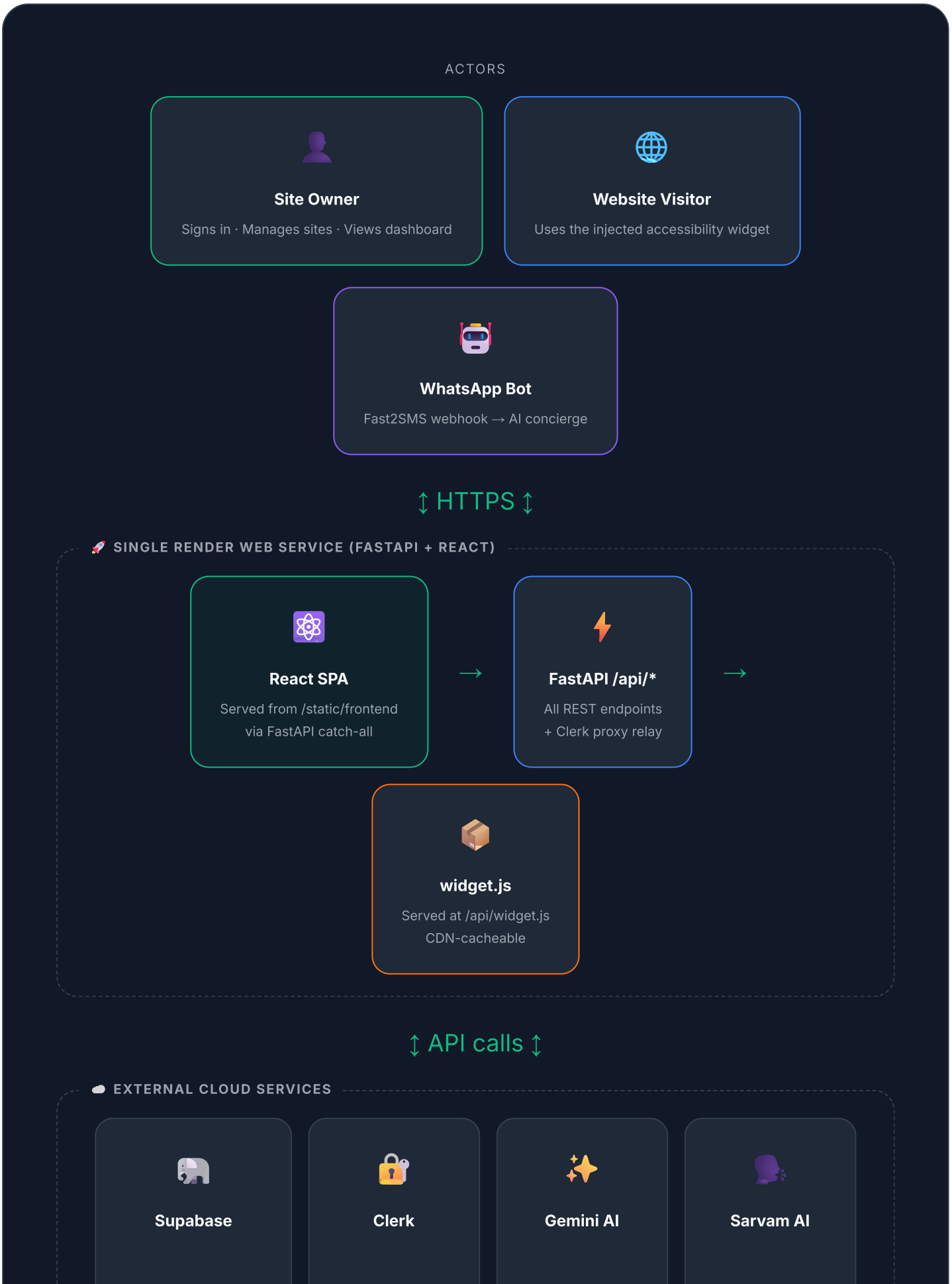
Secure sign-in/sign-up with Clerk, a managed auth SaaS. Supports email/password, Google OAuth, and magic links. The backend verifies Clerk JWTs (RS256) on every protected endpoint. In production, Clerk's Frontend API is proxied through the same domain to avoid third-party cookie issues.

SEO Audit

Alongside accessibility, every server-side audit also checks: title tag, meta description, canonical URL, Open Graph tags, H1 presence, robots.txt, sitemap.xml, page speed signals, mobile viewport, and structured data. Returns an SEO score 0–100 alongside the a11y score.

SECTION 5

How It Works — Architecture



PostgreSQL database
asyncpg pool

Auth / JWT
Sign-in UI

Alt text · Fix tips
Voice commands

Indic STT & TTS
12 languages

Fast2SMS

WhatsApp alerts
Bot webhook

Data Flow — Server-Side Audit

1	2	3	4	5	6
User clicks "Run Audit"	**Audit Engine fetches URL**	**Rules Engine runs**	**Audit saved to DB**	**Dashboard updates**	**Alert check**
Dashboard sends POST /api/sites/{id}/audit with Clerk JWT	httpx GET with SSRF protection, BeautifulSoup parses HTML	12 WCAG checks + SEO checks → score + violations list	asyncpg INSERT into al_audits on Supabase	Score ring, violation list, trend chart shown	Background task: if score dropped ≥5 pts → WhatsApp + email

SECTION 6

User Journey

Landing Page	Sign Up	Add Site	Run Audit	Embed Widget

Enter any URL → free public audit → score + report. No account needed.	→ Clerk email/Google. Auto-redirect to dashboard.	→ Enter site name, URL, optional WhatsApp + email for alerts.	→ One click → full WCAG + SEO report in seconds.	→ Copy <script> tag → paste into site <head> → live toolbar for all visitors.

Embed Code — What Site Owners Copy

```
<!-- AccessLayer: paste into your website <head> -->
<script src="https://your-app.onrender.com/api/widget.js"
  data-site-id="site_abc123xyz"
  data-autofix="true"
  data-sarvam-voice="enabled"
  async></script>
```

Once this tag is live on the site, the widget runs on every page load: scans the live DOM with axe-core rules, reports violations back to AccessLayer, injects the accessibility toolbar, and enables voice navigation for all visitors.

SECTION 7

Tech Stack

Frontend

React 19	Core framework
CRACO	CRA override / Webpack
Tailwind CSS 3	Styling
Radix UI + shadcn	Components
React Router 7	Client routing
@clerk/react	Authentication UI
TanStack Query	Server state
Recharts	Trend charts
Framer Motion	Animations
Axios	HTTP client

Backend

FastAPI 0.110	REST API framework
Uvicorn	ASGI server
asyncpg	Async PostgreSQL driver
httpx	Async HTTP client
BeautifulSoup4 + lxml	HTML parsing
google-genai	Gemini AI SDK
PyJWT	Clerk JWT verification
pydantic 2	Data validation
python-dotenv	Env management
Stripe, Boto3, Motor	Payments / S3 / MongoDB



Supabase

Managed PostgreSQL. 4 tables: sites, audits, alt_cache, wa_log. Auto-schema on startup.



Clerk Auth

Managed auth SaaS. Frontend SDK + backend JWT verification. Production proxy through same domain.



Google Gemini

gemini-3.5-flash-lite for alt text (vision), fix tips (text), voice command parsing, WhatsApp concierge.

SECTION 8

API Reference

Base URL: `https://your-app.onrender.com/api`

Public Endpoints (no auth)

METHOD	PATH	DESCRIPTION
GET	/	Health check — returns service status
GET	/integrations/status	Shows which integrations are configured
POST	/public/scan	Free one-shot WCAG + SEO audit of any URL
GET	/reports/{id}	Fetch a saved audit report (public share link)
GET	/widget.js	The embeddable widget script (JS)
GET	/clerk/config	Returns Clerk publishable key to frontend

Widget Endpoints (site_id key, no user auth)

METHOD	PATH	DESCRIPTION
GET	/widget/{site_id}/config	Widget feature flags for this site
POST	/widget/{site_id}/report	Submit in-browser axe scan results
POST	/widget/{site_id}/alt	Generate AI alt text for an image URL
POST	/widget/{site_id}/tts	Text-to-speech via Sarvam AI

POST	/widget/{site_id}/voice	Upload audio file → STT → voice command
POST	/widget/{site_id}/command	Text transcript → parsed voice command

Dashboard Endpoints (Clerk JWT required)

METHOD	PATH	DESCRIPTION
GET	/me	Current user identity
GET	/sites	List all sites for current user
POST	/sites	Create a new monitored site
GET	/sites/{id}	Get site details
PATCH	/sites/{id}	Update site settings (name, alerts, autofix, etc.)
DELETE	/sites/{id}	Delete site and all its audits
POST	/sites/{id}/audit	Trigger a fresh server-side audit
GET	/sites/{id}/audits	List last 50 audits for a site
POST	/audits/{id}/fix	Get AI fix tip for one violation
POST	/audits/{id}/send-email	Email scorecard to site's alert email
POST	/audits/{id}/send-whatsapp	WhatsApp summary to site's phone
GET	/stats/overview	Totals: sites, audits, avg score, open criticals, trend

Authentication

Send the Clerk session token in the `Authorization` header:

```
Authorization: Bearer <clerk_session_token>
```

SECTION 9

Deployment Guide — Render (Unified)

This project is configured for a single Render Web Service that hosts both the React frontend and the FastAPI backend. No separate services needed. The build script compiles React and places the output inside the backend's static folder; FastAPI then serves it from the same process.

1

Push to GitHub

All files including render.yaml and build_frontend.sh at the repo root

2

New Web Service

render.com → New → Web Service → connect GitHub repo

3

Configure Service

Render auto-reads render.yaml OR set manually (see below)

4

Set Env Vars

Add all secrets from backend/.env in Render dashboard → Environment

5

Deploy

Render builds → assigns URL → you're live!

Manual Render Settings (if not using render.yaml)

SETTING	VALUE
Runtime	Python 3
Build Command	`bash build_frontend.sh && pip install -r backend/requirements.txt`
Start Command	`uvicorn server:app --host 0.0.0.0 --port $PORT --workers 2`
PYTHONPATH	`backend`
Root Directory	*(leave blank — project root)*

How the Single-Server Setup Works

Build Time

1. Render runs `build_frontend.sh`
2. Script installs Node 20 via nvm
3. Runs `npm run build` in `frontend/`
4. Copies `frontend/build/` → `backend/static/frontend/`

Runtime (Request Routing)

1. `/api/*` → FastAPI routes (JSON)
2. `/static/*` → React's hashed JS/CSS assets
3. `/favicon.ico` etc → exact file match
4. Everything else → `index.html` (SPA routing)

5. Renders installs Python deps via pip

Important: Update Clerk Dashboard

After deployment, go to dashboard.clerk.com → Your App → Settings → Domains and add your Render URL (e.g. `https://accesslayer.onrender.com`) as an allowed origin. Without this, sign-in will fail.

SECTION 10

Environment Variables

All secrets live in `backend/.env` locally and in the Render Environment dashboard in production.

VARIABLE	REQUIRED	DESCRIPTION	WHERE TO GET
`SUPABASE_DB_URL`	Yes	PostgreSQL connection string	Supabase → Project → Settings → Database → Connection string (URI mode)
`SUPABASE_POOLER_HOST`	Recommended	PgBouncer pooler host for serverless connections	Supabase → Settings → Database → Connection Pooling
`CLERK_PUBLISHABLE_KEY`	Yes	Public Clerk key (starts with `pk_live_` or `pk_test_`)	dashboard.clerk.com → API Keys
`CLERK_SECRET_KEY`	Yes	Secret Clerk key for JWT verification	dashboard.clerk.com → API Keys
`GEMINI_API_KEY`	Yes	Google Gemini API key	aistudio.google.com → Get API Key

`GEMINI_MODEL`	Optional	Gemini model name (default: gemini-3.5-flash-lite)	—
`SARVAM_API_KEY`	For voice	Sarvam AI key for Indic STT + TTS	dashboard.sarvam.ai
`FAST2SMS_API_KEY`	For WhatsApp	Fast2SMS API key for WhatsApp messaging	fast2sms.com → API
`FAST2SMS_PHONE_NUMBER_ID`	For WhatsApp	Fast2SMS phone number ID	fast2sms.com dashboard
`EMERGENT_EMAIL_KEY`	For email	EmergentAgent transactional email key	emergentagent.com
`EMAIL_FROM_NAME`	Optional	Sender name on emails (default: AccessLayer)	—

SECTION 11

Security Model

🔒 Authentication

- ✓ Clerk JWTs signed RS256, verified server-side on every request
- ✓ No session cookies — stateless Bearer token auth
- ✓ All dashboard routes require `require_user` FastAPI dependency
- ✓ Site ownership checked on every mutation (user_id match)
- ✓ Widget endpoints use site_id as a shared key (no user auth)

🛡️ SSRF Protection

- ✓ Audit engine resolves DNS before making HTTP requests
- ✓ Rejects private IPs: 10.x, 192.168.x, 172.16–31.x, 127.x, ::1
- ✓ Rejects link-local: 169.254.x
- ✓ URL must resolve to a public routable address
- ✓ Request timeout: 30s; max response size enforced

📋 Widget Origin Check

- ✓ Widget `/report` endpoint validates request Origin vs registered site URL
- ✓ Hostname matching (strips www.) prevents other sites from submitting fake reports
- ✓ Demo site bypasses origin check for public testing

🔐 Data Safety

- ✓ No PII stored beyond Clerk user_id (opaque ID)
- ✓ Audit data stored per user, not publicly accessible without audit_id
- ✓ Alt text cache keyed by SHA-256 of image URL, not user
- ✓ All DB queries use parameterized asyncpg statements (no SQL injection)

SECTION 12

Use Cases

🏪 Indian SMB — E-commerce Store

Problem: A Shopify/WooCommerce store has 200+ product images with no alt text, fails keyboard navigation, has poor contrast. They get flagged in an accessibility audit and risk RPwD Act penalties.

Solution: Add AccessLayer widget. AI auto-generates alt text for all images. Dashboard shows exact violations with fix tips. Score jumps from 34 → 91 within a day.

🏥 Healthcare Portal

Problem: A hospital website serves elderly patients, many with visual impairments. Navigation is complex, fonts are small, no voice navigation.

Solution: Widget adds dyslexia mode, large-text toggle, and voice navigation ("Show me appointment booking") in Hindi. WCAG compliance achieved without rewriting any hospital code.

🏛️ Government / NGO Website

Problem: Government portal must comply with GIGW (Guidelines for Indian Government Websites). Manual audits are expensive and infrequent.

Solution: Continuous automated audits catch regressions after every content update. WhatsApp alerts notify the webmaster immediately when score drops. Shareable public report links provide audit evidence.

💻 Web Agency / Freelancer

Problem: A developer builds 20+ client websites and can't afford to manually audit each one for accessibility. Clients ask "are we WCAG compliant?"

Solution: Add all client sites to one AccessLayer account. Dashboard shows all scores at a glance. Email the client a branded scorecard in one click. Offer "accessibility monitoring" as a monthly retainer add-on.

📈 Market Opportunity

2M+	96%	2025
Active Indian business websites	Fail basic WCAG checks	EAA enforcement begins in EU

AccessLayer — Universal Accessibility Layer

Built with FastAPI · React · Google Gemini · Sarvam AI · Supabase · Clerk

Generated October 2026 | v2.0 | Confidential & Proprietary